These can also be combined with field look-ups: `room__name__startswith="R"` filters for a `Venue` with a `Room` with a name starting with `R`.

Sometimes one-to-many relationships just aren't not enough. To allow musicians to belong to several bands, you need many-to-many relationships.

4.7.2 Many-to-many relationships

Foreign keys in a database are a column that contains the ID of a row either in the same or a different table. A `ForeignKey` field in Django constructs one of these columns on the table corresponding to your `Model`. As the column is on the `Model`, you can at most create a one-way relationship. To create a many-to-many relationship, you need an entirely separate table. Technically, these tables contain foreign keys, but in Django, you don't use a `ForeignKey` field to construct them. In fact, you don't have to construct the table at all. The `ManyToManyField` is used to create many-to-many relationships. This field is a little different from the other fields you've used so far. Every other field results in a single column on the object's table. The `ManyToManyField` doesn't result in a new column but a new table.

To be able to relate musicians to bands (both plural), you need a many-to-many relationship. The `ManyToManyField` can go on either the `Musician` or `Band` class. When Django sees this kind of field, it creates a relationship table with three columns: a unique ID for the relationship and a column for each of the two keys of the related objects. The table is named based on the app and the two models participating in the relationship.

Besides establishing my age and musical preferences, figure 4.10 shows three tables, for musicians, bands, and their many-to-many relationship. Famous guitarist

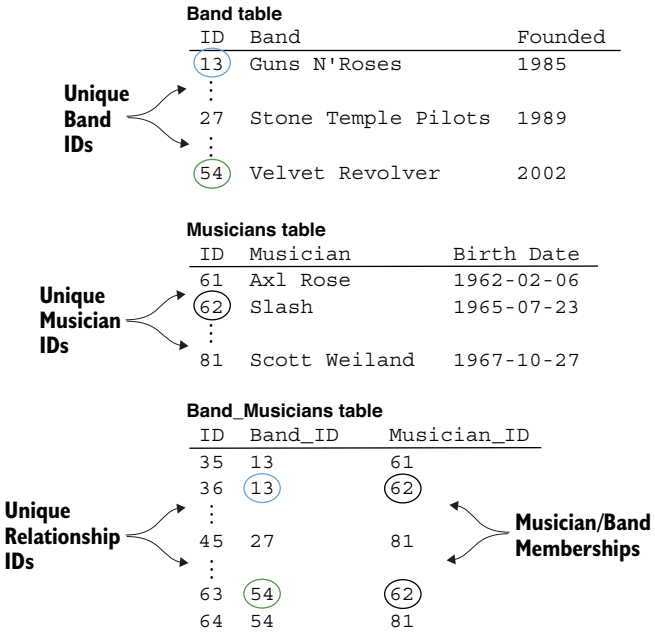


Figure 4.10 Database tables for many-to-many relationship between bands and musicians